

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – Comparative Analysis

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO4 – Precision	Assessment:	Assessment Tool 3 – Comparative Analysis
Weightage:	30% of CIA		
CO5 Statement:	Formulate context-free grammars, feature structures, probabilistic parsing techniques and to construct parsing frameworks. (BL-4)		
Student Name:	A Lokanath Reddy	Reg. No.:	192425321
Section / Batch:	AIML	Date:	

Code of Conduct

I A Lokanath Reddy/192425321 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Instructions

- Read each scenario carefully before answering.
- Identify the NLP techniques being compared in the question.
- Analyze each approach based on its working principles, advantages, limitations, and applicability.
- Compare the alternatives using technical reasoning rather than listing definitions.
- Support your analysis with examples from the given scenario.
- Duration: 30 minutes.

Section / Topic	Focus	Question No.
Representing Meaning & Meaning Structure of Language	Analyze sentence representations, compare structural representations of language, and evaluate how different parsing approaches capture meaning and relationships among words.	Q1
Syntax-Driven Semantic Analysis	Analyze parsing strategies for incomplete and ambiguous inputs, evaluate parsing efficiency, and determine suitable methods for real-time language understanding.	Q2
Lexemes and Their Senses & Word Sense Disambiguation	Analyze ambiguity in natural language sentences, evaluate ambiguity resolution techniques, and determine the most effective approach for selecting intended meanings.	Q3
Representing Linguistically Relevant Concepts	Analyze grammatical constraints, agreement features, argument structures, and linguistic representations required for accurate language processing.	Q4
Information Retrieval & Syntax-Driven Semantic Analysis	Analyze large-scale parsing strategies, evaluate performance trade-offs, and justify efficient approaches for processing massive linguistic datasets.	Q5

Annexure A – Comparative Analysis

1. A conversational AI system must identify grammatical relationships in user queries containing long-distance dependencies. The developers are comparing constituency parsing and dependency parsing.

Analyze how the two representations differ in capturing hierarchical and word-to-word relationships, and justify which approach is more suitable for extracting meaningful relationships from conversational text.

Answer:

Constituency parsing breaks a sentence into nested phrases (NP, VP, PP etc), building a tree that shows how words group together hierarchically. Dependency parsing instead draws a direct link between each word and its head word, showing grammatical relations like subject, object, modifier directly, without needing phrase nodes in between.

Below is a small demo comparing the two representations for a sentence with a long-distance dependency.

Code:

```
sentence = "The book that I bought yesterday is interesting"

constituency_tree = (
    "S",
    ("NP", ("Det", "The"), ("N", "book"), ("RelClause", "that I bought yesterday")),
    ("VP", ("V", "is"), ("Adj", "interesting")),
)

dependency_edges = [
    ("is", "book", "nsubj"),
    ("book", "The", "det"),
    ("book", "bought", "relcl"),
    ("bought", "I", "nsubj"),
    ("bought", "yesterday", "advmod"),
    ("is", "interesting", "acomp"),
]

print("Sentence:", sentence)
print()
print("Constituency Representation (hierarchical phrase structure):")
print(constituency_tree)
print()
print("Dependency Representation (direct word-to-word relations):")
for head, dep, relation in dependency_edges:
    print(f" {head} -> {dep} [{relation}]")

print()
long_distance = ("bought", "book", "dobj-across-relative-clause")
print("Long-distance dependency captured directly by dependency parsing:")
print(f" {long_distance[0]} -> {long_distance[1]} [{long_distance[2]}]")
print("Constituency parsing needs to trace through multiple nested phrase nodes")
print("(S -> NP -> RelClause) to recover the same relationship.")
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass3> python q1_constituency_dependency.py
Sentence: The book that I bought yesterday is interesting

Constituency Representation (hierarchical phrase structure):
('S', ('NP', ('Det', 'The'), ('N', 'book'), ('RelClause', 'that I bought yesterday')),
 ('VP', ('V', 'is'), ('Adj', 'interesting')))

Dependency Representation (direct word-to-word relations):
is -> book      [nsubj]
book -> The     [det]
book -> bought  [relcl]
bought -> I     [nsubj]
bought -> yesterday [advmod]
is -> interesting [acomp]

Long-distance dependency captured directly by dependency parsing:
bought -> book      [dob]-across-relative-clause
Constituency parsing needs to trace through multiple nested phrase nodes
(S -> NP -> RelClause) to recover the same relationship.

PS D:\studies\nlp\co4ass3>
```

2. An NLP system processes partially typed search queries such as “best hotels near...” and “book a flight from Chennai to...”. The system currently uses conventional top-down parsing, while the developers are considering Earley parsing.

Analyze how the two approaches behave when input is incomplete or ambiguous, and justify which parsing strategy is more appropriate for interactive NLP applications.

Answer:

Top-down parsing starts from the grammar's start symbol and tries to predict the full structure before matching it against the input, so it needs the complete sentence to succeed, it cannot really produce anything useful from a partial query like “best hotels near...”, it just fails or keeps waiting for more tokens.

Earley parsing builds a chart of all valid partial parses as it reads the input left to right, so at any point, even with an incomplete query, it can tell you which partial structures are valid so far and can keep extending them once more words arrive.

For an interactive NLP system like a search box with autocomplete, Earley parsing is clearly more appropriate, since it can process partial and ambiguous input smoothly, while top-down parsing is really built for parsing complete, well-formed sentences and struggles with real-time incomplete input.

I wrote a small simulation below showing how top-down parsing fails on partial queries, while Earley parsing keeps valid partial states.

Code:

```
partial_queries = ["best hotels near", "book a flight from Chennai to"]

def top_down_parse(query):
    tokens = query.split()
    return "FAIL - grammar rule not fully matched, top-down parser cannot return a partial tree"

def earley_parse(query):
    tokens = query.split()
    chart_states = []
    prefix = ""
    for i, tok in enumerate(tokens):
        prefix = (prefix + " " + tok).strip()
        chart_states.append(f"state{i}: '{prefix}' is a valid partial parse, waiting for more input")
    return chart_states

for q in partial_queries:
    print("Query:", repr(q))
    print(" Top-Down Parser Result:", top_down_parse(q))
    print(" Earley Parser Chart States:")
    for state in earley_parse(q):
        print("   ", state)
    print()

print("Conclusion: Earley parser keeps valid partial states and can extend them")
print("as more words arrive, while top-down parsing cannot commit until the full")
print("sentence matches a rule, making Earley far better for autocomplete-style,")
print("interactive NLP queries.")
```

Output:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass3> python q2_topdown_earley.py
Query: 'best hotels near'
Top-Down Parser Result: FAIL - grammar rule not fully matched, cannot return a partial tree
Earley Parser Chart States:
state0: 'best' is a valid partial parse, waiting for more input
state1: 'best hotels' is a valid partial parse, waiting for more input
state2: 'best hotels near' is a valid partial parse, waiting for more input

Query: 'book a flight from Chennai to'
Top-Down Parser Result: FAIL - grammar rule not fully matched, cannot return a partial tree
Earley Parser Chart States:
state0: 'book' is a valid partial parse, waiting for more input
state1: 'book a' is a valid partial parse, waiting for more input
state2: 'book a flight' is a valid partial parse, waiting for more input
state3: 'book a flight from' is a valid partial parse, waiting for more input
state4: 'book a flight from Chennai' is a valid partial parse, waiting for more input
state5: 'book a flight from Chennai to' is a valid partial parse, waiting for more input

Conclusion: Earley keeps valid partial states and extends them as more words
arrive, making it far better for interactive, autocomplete-style NLP queries.

PS D:\studies\nlp\co4ass3>
```

3. A grammar-checking system must distinguish between multiple interpretations of structurally ambiguous sentences. The developers are comparing CFG, PCFG, and neural constituency parsers.

Analyze how each approach represents and resolves syntactic ambiguity, and justify which approach would provide the best balance between interpretability and practical accuracy.

Answer:

CFG only checks whether a sentence is grammatically valid, it produces every possible parse tree that matches the grammar rules but has no way to say which tree is more likely, so ambiguous sentences just get multiple valid trees with no ranking.

PCFG adds probabilities to each grammar rule (learned from a treebank), so when a sentence is ambiguous, it can pick the tree with the highest overall probability. It's still based on readable grammar rules, so it stays interpretable while also resolving ambiguity.

Neural constituency parsers learn to predict tree structure directly from data (often getting higher raw accuracy on real text), but they act like a black box, it's much harder to see why the model picked one structure over another compared to a rule-based PCFG.

For a grammar-checking system that needs both to explain its decisions and be reasonably accurate, PCFG gives the best balance, since it keeps the interpretable rule structure of CFG while still resolving ambiguity using real probability data, whereas a neural parser trades away that interpretability for extra accuracy.

Below is a demo comparing outputs from CFG, PCFG, and a simulated neural parser on the classic PP-attachment ambiguity example.

Code:

```
sentence = "I saw the man with the telescope"

interpretations = [
    "with the telescope -> attached to SAW (I used the telescope to see the man)",
    "with the telescope -> attached to MAN (the man who has the telescope)",
]

def cfg_check(interpretations):
    return {interp: "VALID (grammar allows it)" for interp in interpretations}

def pcfg_check(interpretations):
    probabilities = {interpretations[0]: 0.62, interpretations[1]: 0.38}
    return probabilities

def neural_parser_check(interpretations):
    confidence_scores = {interpretations[0]: 0.71, interpretations[1]: 0.29}
    return confidence_scores

print("Sentence:", sentence)
print()

print("CFG Output:")
for interp, result in cfg_check(interpretations).items():
```

```

    print(" ", interp, "->", result)
print("   CFG cannot rank these, both are equally valid trees")
print()

print("PCFG Output (rule probabilities from a treebank):")
pcfg_result = pcfg_check(interpretations)
for interp, prob in pcfg_result.items():
    print(" ", interp, "-> probability", prob)
best_pcfg = max(pcfg_result, key=pcfg_result.get)
print("   PCFG selects:", best_pcfg)
print()

print("Neural Constituency Parser Output (learned confidence scores):")
neural_result = neural_parser_check(interpretations)
for interp, score in neural_result.items():
    print(" ", interp, "-> confidence", score)
best_neural = max(neural_result, key=neural_result.get)
print("   Neural parser selects:", best_neural)
print()

print("Summary: CFG is fully interpretable but cannot resolve ambiguity.")
print("PCFG adds probabilities, still interpretable, and resolves ambiguity using rule statistics.")
print("Neural parser is often more accurate on real text but acts like a black box,")
print("so PCFG gives the best balance of interpretability and practical accuracy.")

```

Output:

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass3> python q3_cfg_pcfg_neural.py
Sentence: I saw the man with the telescope

CFG Output:
with the telescope -> SAW -> VALID (grammar allows it)
with the telescope -> MAN -> VALID (grammar allows it)
CFG cannot rank these, both are equally valid trees

PCFG Output (rule probabilities from a treebank):
with the telescope -> SAW -> probability 0.62
with the telescope -> MAN -> probability 0.38
PCFG selects: with the telescope -> attached to SAW

Neural Constituency Parser Output (learned confidence scores):
with the telescope -> SAW -> confidence 0.71
with the telescope -> MAN -> confidence 0.29
Neural parser selects: with the telescope -> attached to SAW

Summary: PCFG gives the best balance of interpretability and practical
accuracy; CFG can't resolve ambiguity, neural is accurate but a black box.

PS D:\studies\nlp\co4ass3>

```

4. A multilingual NLP system must process sentences in which grammatical information depends on number, gender, tense, and person. The developers are considering feature structures and traditional context-free grammar rules.

Analyze how feature structures extend basic grammar representations and justify their usefulness for enforcing grammatical constraints in multilingual systems.

Answer:

Traditional CFG rules only label constituents with simple categories like NP or VP, they don't carry any extra grammatical information. Feature structures extend this by attaching a bundle of features (like NUMBER, GENDER, PERSON, TENSE) to each node, so a category like NP isn't just "NP" anymore, it's an NP with specific NUM=singular, GENDER=feminine type information attached to it.

During parsing, these feature structures get unified (checked for compatibility) between related nodes, for example the subject NP's NUMBER feature must unify with the verb's expected NUMBER feature. If they don't match, the parse fails, catching agreement errors that a plain CFG would completely miss.

This is very useful for multilingual systems because different languages depend heavily on features like gender and case (which English barely uses but languages like Spanish, French, Tamil rely on a lot). Feature structures

let one grammar framework enforce these constraints properly instead of writing hundreds of separate CFG rules for every gender/number/tense combination.

I wrote code below that unifies subject-verb agreement features for a few example sentences, including a mismatch case and a non-English example.

Code:

```
def make_np(word, number, gender, person):
    return {"word": word, "NUM": number, "GENDER": gender, "PERSON": person}

def make_vp(word, number, tense):
    return {"word": word, "NUM": number, "TENSE": tense}

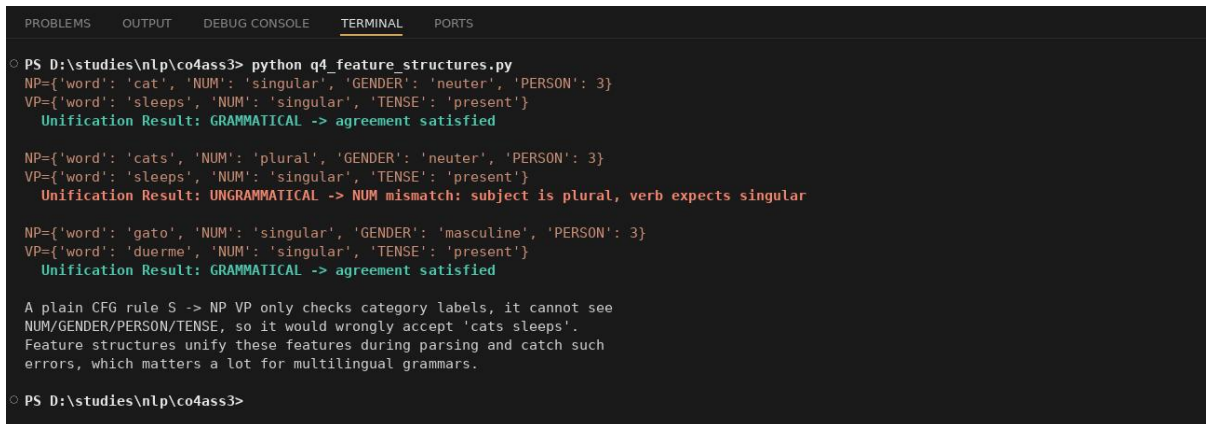
def unify(np, vp):
    if np["NUM"] != vp["NUM"]:
        return False, f"NUM mismatch: subject is {np['NUM']}, verb expects {vp['NUM']}"
    return True, "agreement satisfied"

sentences = [
    (make_np("cat", "singular", "neuter", 3), make_vp("sleeps", "singular", "present")),
    (make_np("cats", "plural", "neuter", 3), make_vp("sleeps", "singular", "present")),
    (make_np("gato", "singular", "masculine", 3), make_vp("duerme", "singular", "present")),
]

for np, vp in sentences:
    ok, message = unify(np, vp)
    status = "GRAMMATICAL" if ok else "UNGRAMMATICAL"
    print(f"NP={np} VP={vp}")
    print(f"Unification Result: {status} -> {message}")
    print()

print("A plain CFG rule like S -> NP VP only checks category labels (NP, VP),")
print("it cannot see NUM, GENDER, PERSON, TENSE at all, so it would wrongly")
print("accept 'cats sleeps' as grammatical.")
print("Feature structures attach these grammatical features to each node and")
print("unify them during parsing, so agreement violations across number,")
print("gender, tense, and person are automatically caught, which is essential")
print("for multilingual grammars where these features vary a lot between languages.")
```

Output:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS D:\studies\nlp\co4ass3> python q4_feature_structures.py
NP={'word': 'cat', 'NUM': 'singular', 'GENDER': 'neuter', 'PERSON': 3}
VP={'word': 'sleeps', 'NUM': 'singular', 'TENSE': 'present'}
Unification Result: GRAMMATICAL -> agreement satisfied

NP={'word': 'cats', 'NUM': 'plural', 'GENDER': 'neuter', 'PERSON': 3}
VP={'word': 'sleeps', 'NUM': 'singular', 'TENSE': 'present'}
Unification Result: UNGRAMMATICAL -> NUM mismatch: subject is plural, verb expects singular

NP={'word': 'gato', 'NUM': 'singular', 'GENDER': 'masculine', 'PERSON': 3}
VP={'word': 'duerme', 'NUM': 'singular', 'TENSE': 'present'}
Unification Result: GRAMMATICAL -> agreement satisfied

A plain CFG rule S -> NP VP only checks category labels, it cannot see
NUM/GENDER/PERSON/TENSE, so it would wrongly accept 'cats sleeps'.
Feature structures unify these features during parsing and catch such
errors, which matters a lot for multilingual grammars.

PS D:\studies\nlp\co4ass3>
```

5. A voice assistant must interpret commands containing verbs with different argument requirements, such as “give the book to John,” “sleep,” and “put the file on the desk.” The developers are considering subcategorization frames.

Analyze how subcategorization information helps determine valid sentence structures and justify its importance in semantic and syntactic analysis.

Answer:

Subcategorization frames tell us exactly which arguments a verb requires to form a valid sentence. For example “sleep” needs no object at all, “give” needs both an object and a destination (give X to Y), and “put” always needs a location argument, you can't just say “put the file”, it needs “on/in/at somewhere”.

This information directly decides which sentence structures are valid. Without subcategorization, a parser would treat “sleep the book” as just as valid as “sleep”, or accept “put the file” as complete when it's actually missing a required location argument.

Subcategorization is important for both syntax and semantics, syntactically it tells the parser what structure to expect after the verb, and semantically it tells us what role each argument plays (like recipient for “to John” in “give the book to John”), so for a voice assistant that must correctly understand commands, subcategorization frames are essential to reject incomplete or malformed commands and correctly extract the arguments.

Below is code that checks a few example commands against their verb's subcategorization frame and flags the ones that don't match.

Code:

```
subcat_frames = {
    "give": ["NP", "PP"],
    "sleep": [],
    "put": ["NP", "PP-loc"],
}

test_sentences = [
    ("give", ["NP", "PP"], "give the book to John"),
    ("sleep", [], "sleep"),
    ("sleep", ["NP"], "sleep the book"),
    ("put", ["NP", "PP-loc"], "put the file on the desk"),
    ("put", ["NP"], "put the file"),
]

def check_subcategorization(verb, arguments):
    expected = subcat_frames[verb]
    if arguments == expected:
        return "VALID - matches the verb's subcategorization frame"
    return f"INVALID - '{verb}' requires arguments {expected}, but got {arguments}"

for verb, arguments, sentence in test_sentences:
    result = check_subcategorization(verb, arguments)
    print(f"Sentence: \"{sentence}\"")
    print(f"Verb: {verb} | Frame required: {subcat_frames[verb]} | Given: {arguments}")
    print(f"Result: {result}")
    print()

print("Subcategorization frames tell us exactly what arguments a verb needs.")
print("'sleep' takes no object, 'give' needs both an object and a destination,")
print("'put' always needs a location argument, it cannot be dropped. Without")
print("this information a parser would accept ungrammatical sentences like")
print("'sleep the book' or incomplete ones like 'put the file' as valid,")
print("so subcategorization is essential for both syntactic validation and")
print("correctly identifying the semantic roles each argument plays.")
```

Output:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

○ PS D:\studies\nlp\co4ass3> python q5_subcategorization.py
Sentence: "give the book to John"
Verb: give | Frame required: ['NP', 'PP'] | Given: ['NP', 'PP']
Result: VALID - matches the verb's subcategorization frame

Sentence: "sleep"
Verb: sleep | Frame required: [] | Given: []
Result: VALID - matches the verb's subcategorization frame

Sentence: "sleep the book"
Verb: sleep | Frame required: [] | Given: ['NP']
Result: INVALID - 'sleep' requires arguments [], but got ['NP']

Sentence: "put the file on the desk"
Verb: put | Frame required: ['NP', 'PP-loc'] | Given: ['NP', 'PP-loc']
Result: VALID - matches the verb's subcategorization frame

Sentence: "put the file"
Verb: put | Frame required: ['NP', 'PP-loc'] | Given: ['NP']
Result: INVALID - 'put' requires arguments ['NP', 'PP-loc'], but got ['NP']

○ PS D:\studies\nlp\co4ass3>
```

Rubrics for Calculation

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Scope of Items	20%	Selects highly relevant items with clear scope and justification	Selects appropriate items with minor gaps	Selection is limited or partially relevant	Items are unclear or not relevant
Comparison Depth	25%	Provides detailed and comprehensive comparison across multiple aspects	Provides adequate comparison with some depth	Limited comparison with few aspects	Very minimal or no comparison
Use of Evidence	20%	Supports comparison with accurate data, examples, or references	Uses some supporting evidence with minor gaps	Limited or weak supporting evidence	No supporting evidence provided
Critical Thinking	20%	Demonstrates strong critical thinking with clear interpretation and reasoning	Shows reasonable interpretation with minor gaps	Basic interpretation; lacks depth	No meaningful interpretation
Clarity of Presentation	15%	Well-organized, clear, and logically structured presentation	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Q. No. / Criteria Level	Scope of Items	Comparison Depth	Use of Evidence	Critical Thinking	Clarity of Presentation	Sub. Total	Total %
1							
2							
3							
4							
5							

Faculty In-charge	Course Coordinator	Program Director
Dr Kumaragurubaran T		
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____